# Rasin MVP Specification

Version 1.0 · October 14, 2026 · Author: Asem Al-Zaghal

## 1. Goal

Prove one loop works and is trusted: **import transactions → AI categorizes → user approves → app calculates the federal tax estimate.**

Two features only: **Categorization** (AI) and **Tax calculation** (deterministic code, no AI). The other screens (dashboard, chart of accounts, assistant, profile and billing) only support these two.

## 2. Locked scope and assumptions

| Item | Decision |
|---|---|
| Tax year | 2026 |
| State | **Texas** (no state personal income tax, so federal tax only) |
| Taxpayer | Single filer, sole proprietor or single-member LLC (Schedule C) |
| Accounting basis | Cash basis |
| Deduction | Standard deduction ($16,100) |
| Income | Business income only (no W-2, no other income) |
| Credits | None |
| Data source | CSV upload only (no Plaid yet) |
| Users | One user per account |

**Out of scope (do not build):** QBI deduction (the estimate will run slightly high, so say so in the UI), Additional Medicare Tax (show an "unsupported" banner if net profit exceeds $200,000), state, sales and franchise tax, home office, depreciation, mileage rate, invoices, customers, reports beyond a simple summary, mobile app, admin panel.

# 3. Feature 1: Categorization

## 3.1 Input

CSV with columns: `date`, `description`, `amount` (negative = money out), optional `merchant`. Reject rows with a missing date or amount and show them in an error list.

## 3.2 Fixed category list

The AI may only choose from this list. Each category has a deductible percentage used by the tax feature.

| Category | Deductible |
|---|---|
| Income | n/a (revenue) |
| Advertising | 100% |
| Car & truck (actual expenses, business use only) | 100% |
| Contract labor | 100% |
| Insurance (business) | 100% |
| Legal & professional services | 100% |
| Office expense | 100% |
| Rent / lease | 100% |
| Repairs & maintenance | 100% |
| Supplies | 100% |
| Software & subscriptions | 100% |
| Travel | 100% |
| Meals | **50%** |
| Utilities & phone | 100% |
| Bank & processing fees | 100% |
| Taxes & licenses | 100% |
| Owner draw / transfer | Excluded |

| Category | Deductible |
|---|---|
| Personal | Excluded |
| Uncategorized | Excluded until categorized |

The user is responsible for marking personal-use items as Personal.

## 3.3 Pipeline

1. Parse and normalize the CSV (trim, lowercase merchant, strip store numbers).
2. **Rules first:** if a user rule matches (`merchant contains X → category Y`), use it with confidence 1.00 and reason "Your rule".
3. **AI for the rest:** send unmatched rows to the Claude API in batches (up to 25 per call).
4. Validate the response (valid JSON, category exists in the list, confidence between 0 and 1). On failure retry once, then mark `Uncategorized` with confidence 0.
5. Save each result as a **suggestion**. Nothing is posted or counted yet.
6. The user approves, edits, or rejects in the Review Queue.

## 3.4 AI contract

- **Model:** start with `claude-haiku-4-5-20251001`; compare against `claude-sonnet-5-5` on your labeled test set and keep the cheaper one if accuracy is equal.
- **Temperature:** 0.
- **Send only:** date, description, merchant, amount, direction. Never send account numbers, names, or emails.
- **Output per transaction (JSON only):** `{ "id", "category", "confidence", "reason" }` where `reason` is at most 20 words.
- **Instruction:** choose only from the provided category list; if unsure, return `Uncategorized` with low confidence; never invent amounts or categories.

## 3.5 Confidence tiers

| Confidence | Label | UI |
|---|---|---|
| 0.90 and above | High | Green bar |
| 0.70 to 0.89 | Medium | Amber bar |

| Confidence | Label | UI |
|---|---|---|
| Below 0.70 | Low | Red bar and "Please check" |

**No auto-approve in the MVP.** Every transaction needs an explicit approval.

### 3.6 Learning from edits

When the user changes a category, offer "Always categorize this merchant as X". Saving creates a rule. Rules always beat AI.

### 3.7 States

`imported → suggested → approved` (or `edited → approved`). Only **approved** transactions count in any total or tax figure. Every state change is written to the audit log.

## 4. Feature 2: Tax calculation (deterministic, no AI)

All figures are computed in code from approved transactions. The AI is never involved. Money is stored as integer cents and every step rounds half-up to the cent.

### 4.1 Parameters (tax-2026.json, versioned config, not hard-coded)

| Parameter | Value |
|---|---|
| Standard deduction (single) | $16,100 |
| SE earnings factor | 92.35% |
| Social Security rate / wage base | 12.4% up to $184,500 |
| Medicare rate | 2.9% (no cap) |
| SE tax minimum | Net SE earnings under $400 → SE tax is $0 |

**2026 brackets, single filer (taxable income):**

| Rate | From | To |
|---|---|---|
| 10% | $0 | $12,400 |
| 12% | $12,400 | $50,400 |

| Rate | From | To |
|---|---|---|
| 22% | $50,400 | $105,700 |
| 24% | $105,700 | $201,775 |
| 32% | $201,775 | $256,225 |
| 35% | $256,225 | $640,600 |
| 37% | $640,600 | no limit |

Source: IRS Revenue Procedure 2025-32 (brackets, standard deduction) and the Social Security wage base of $184,500 for 2026. Re-check every figure against IRS.gov before launch and have a CPA review the formulas once.

## 4.2 Formula

1. `revenue` = sum of approved Income transactions.
2. `expenses` = sum of approved expense amounts × the category's deductible %.
3. `net_profit` = revenue − expenses. If it is 0 or less, every tax figure is $0.
4. `se_earnings` = net_profit × 0.9235.
5. `se_tax` = 0 if se_earnings < $400; otherwise 12.4% × min(se_earnings, $184,500) + 2.9% × se_earnings.
6. `agi` = net_profit − (se_tax ÷ 2).
7. `taxable_income` = max(0, agi − $16,100).
8. `income_tax` = progressive brackets applied to taxable_income (tax each slice at its own rate).
9. `total_tax` = income_tax + se_tax.

Show a banner if net_profit exceeds $200,000: "Above this level the estimate is not supported yet."

## 4.3 Quarterly estimate

Inputs: year-to-date net profit (Jan 1 to today) and `payments_made` (entered by the user).

- **Tax set aside so far** = total_tax computed on YTD net profit.
- **Suggested next payment** = max(0, tax set aside so far − payments_made).

- **Projected year-end tax** (informational) = total_tax computed on YTD net profit × 365 ÷ days elapsed.
- **Due dates:** Apr 15, Jun 15, Sep 15, and Jan 15 of the following year. Store them in config.

Show "Estimate only. Not tax advice." on the page.

### 4.4 Test cases (must match to the cent)

| Net profit | SE earnings | SE tax | Taxable income | Income tax | Total tax |
|---|---|---|---|---|---|
| $300 | $277.05 | $0.00 | $0.00 | $0.00 | $0.00 |
| $60,000 | $55,410.00 | $8,477.73 | $39,661.13 | $4,511.34 | $12,989.07 |
| $120,000 | $110,820.00 | $16,955.46 | $95,422.27 | $15,704.90 | $32,660.36 |

Add edge tests: profit of exactly $0, negative profit, profit that lands exactly on a bracket edge, and a 50% meals expense.

## 5. Data model (MySQL 8)

- `users` (id, name, email, password, business_name, created_at)
- Cashier tables ( `subscriptions` , `subscription_items` ) for the single Stripe plan
- `accounts` : the chart of accounts (id, user_id, code, name, type: asset / liability / equity / revenue / expense, category_id nullable, is_system). Seeded per user from a default template; every income and expense category maps to one account.
- `categories` (id, name, deductible_pct, kind: income / expense / excluded, default_account_id)
- `import_batches` (id, user_id, filename, bank_account_id, row_count, created_at). `bank_account_id` points to an asset account such as "Business Checking".
- `transactions` (id, user_id, import_batch_id, date, description, merchant, amount_cents BIGINT, status, category_id)
- `suggestions` (id, transaction_id, category_id, confidence DECIMAL(3,2), reason, model, created_at)
- `rules` (id, user_id, merchant_contains, category_id)
- `journal_entries` (id, user_id, transaction_id, date, description) and `journal_lines` (id, entry_id, account_id, debit_cents, credit_cents)
- `tax_payments` (id, user_id, date, amount_cents)

- `chat_messages` (id, user_id, role: user / assistant, content, created_at)
- `audit_log` (id, user_id, actor: user / ai / system, action, object_type, object_id, before, after, created_at). Append-only.

**The ledger is now required, because the chart of accounts exists.** Approving a transaction creates one balanced journal entry inside a single database transaction. Money out: debit the expense account, credit the bank account. Money in: debit the bank account, credit the revenue account. Owner draw / transfer and Personal post to an Owner's draw equity account.

**MySQL notes:** use InnoDB and utf8mb4; store money as BIGINT cents. A CHECK constraint cannot compare debits and credits across rows, so enforce "debits = credits" in the posting service and cover it with a test. Optionally add a trigger that blocks UPDATE and DELETE on posted journal lines. Use JSON columns only for the audit log's before and after values.

## 6. Screens (9)

1. **Login / sign up** (Laravel Sanctum)
2. **Dashboard:** year-to-date revenue, expenses, net profit, tax set aside so far, the number of items waiting in the Review Queue, and a monthly profit chart. Approved transactions only.
3. **Import:** upload a CSV, choose the bank account, see the row count and an error list.
4. **Review Queue:** merchant, amount, suggested category (editable dropdown), confidence bar, one-line reason, Approve / Edit / Reject, a banner "Nothing counts until you approve", and a bulk "Approve all High confidence" button.
5. **Transactions:** searchable list of all transactions with status and category.
6. **Tax Estimate:** the figures from section 4, the next due date, a payment entry form, and the disclaimer.
7. **Chart of Accounts:** accounts grouped by type with balances from the journal lines; add or rename custom accounts; system accounts cannot be deleted; click an account to see its lines.
8. **AI Assistant:** chat about the user's own numbers (rules in 6.1).
9. **Profile (settings and billing):** two tabs. Settings: name, business name, email, password, and the fixed tax profile (2026, single filer, Texas) shown read-only. Billing: the single plan, subscribe or cancel, payment method and invoices through the Stripe customer portal (Laravel Cashier).

### 6.1 AI assistant rules

- **Read-only.** It cannot approve, edit, or post anything.
- **It never calculates.** It answers by calling fixed backend functions through Claude tool use (`get_summary(period)`, `get_category_totals(period)`, `get_tax_estimate()`, `list_transactions(filters)`) and explains the numbers they return.
- **Every answer names its data,** for example "Based on 142 approved transactions, Jan 1 to Oct 14, 2026", and says when pending transactions are excluded.
- **Tax questions** stay inside the estimate in section 4, with the disclaimer. No other tax advice.
- **Safety limits:** every function filters by the logged-in user's id, the last 10 messages are sent as context, and requests are rate-limited per user.

## 7. API (minimum)

- `POST /api/register`, `POST /api/login`, `POST /api/logout`
- `GET /api/dashboard`
- `POST /api/imports` (queues the categorization job), `GET /api/imports/{id}`
- `GET /api/transactions?status=&q=`, `PATCH /api/transactions/{id}`, `POST /api/transactions/approve-bulk`
- `GET /api/rules`, `POST /api/rules`, `DELETE /api/rules/{id}`
- `GET /api/accounts`, `POST /api/accounts`, `PATCH /api/accounts/{id}`, `GET /api/accounts/{id}/lines`
- `GET /api/tax/estimate?year=2026`, `GET /api/tax/payments`, `POST /api/tax/payments`
- `GET /api/assistant/messages`, `POST /api/assistant/messages`
- `GET /api/profile`, `PATCH /api/profile`, `GET /api/billing` (Cashier: subscribe, cancel, portal link)
- `GET /api/audit-log`

## 8. Stack

Laravel (current release, PHP 8.3 or newer) as the API, a React (JavaScript) single-page app, and MySQL 8. Sanctum handles SPA login, Laravel queues (database driver first) run the categorization job, Laravel Cashier handles Stripe billing, and Form Requests validate

input. The Anthropic API is called from a Laravel service class. The tax calculator is a plain PHP class with no framework, database, or network dependency, tested with PHPUnit or Pest. Never use PHP floats for money: use integer cents and an exact-math helper (brick/math or bcmath) with explicit half-up rounding so the section 4.4 test cases pass to the cent.

## 9. Definition of done

- All test cases in 4.4 pass to the cent in automated PHP tests.
- On a labeled set of at least 200 of your own transactions, the AI's top suggestion matches your label at least 85% of the time.
- Unapproved transactions never affect any total, tax figure, account balance, or assistant answer.
- Every approval creates one journal entry whose debits equal credits, and a failed posting leaves nothing behind.
- Every state change appears in the audit log.
- A 1,000-row CSV is categorized in the background queue in under 60 seconds (target), and the UI never blocks.
- The AI output validator rejects any category outside the list.
- The assistant never states a number that is not in a function result (spot-check 20 questions).
- A user can subscribe, cancel, and see invoices in Stripe test mode.

## 10. Build order

1. Tax calculator class and its tests (no UI).
2. Laravel and MySQL setup, migrations, category and default chart-of-accounts seeds, Sanctum auth, React shell with login / sign up.
3. CSV import and the queue job.
4. Categorization service, rules, and the labeled test set.
5. Review Queue and Transactions screens, with the posting service that writes journal entries.
6. Tax Estimate, Dashboard, and Chart of Accounts screens.
7. AI Assistant (backend functions first, chat screen second).
8. Profile with Billing (Cashier, Stripe test mode), audit log, polish.

## 11. Open decisions

- Haiku vs. Sonnet for categorization (decide from the accuracy test).
- When to add the QBI deduction (it lowers the estimate for most users).
- Plaid sync and a second state after the MVP proves the loop.